

Практическая работа №6: Приоритеты и сортировка задач

Цель работы

Добавить в приложение «Список дел» возможность задавать приоритет задачи (высокий, средний, низкий) и сортировать задачи по приоритету.

1. Что изменится в хранении задач

Было:

```
{
  "text": "Купить хлеб",
  "date": "2026-05-04",
  "done": false
}
```

Станет:

```
{
  "text": "Купить хлеб",
  "date": "2026-05-04",
  "done": false,
  "priority": "средний"
}
```

Варианты приоритета: "высокий", "средний", "низкий".

2. Подготовка: удаляем старый файл

Поскольку структура данных изменилась (добавилось поле `priority`), старый `tasks.json` нам не подходит.

1. Остановите приложение (`Ctrl + C` в терминале)
2. Удалите файл `tasks.json`:
 - В терминале: `rm tasks.json`
 - Или удалите вручную через файловый менеджер

При следующем запуске приложение создаст новый `tasks.json` с правильной структурой.

3. Изменяем код

3.1. Добавляем выбор приоритета в форму добавления

Файл `templates/index.html`. Найдите форму добавления задачи и добавьте выпадающий список `<select>` :

```
<form method="POST" action="/add">
    <input type="text" name="task" placeholder="Новая задача" required>

    <select name="priority">
        <option value="высокий">🔴 Высокий</option>
        <option value="средний" selected>🟡 Средний</option>
        <option value="низкий">🟢 Низкий</option>
    </select>

    <button type="submit">Добавить</button>
</form>
```

3.2. Изменяем маршрут `/add` в `app.py`

Теперь нужно сохранять приоритет вместе с задачей:

```
@app.route('/add', methods=['POST'])
def add_task():
    new_task = request.form.get('task')
    priority = request.form.get('priority', 'средний') # по умолчанию сред

    if new_task:
        today = date.today().strftime('%Y-%m-%d')
        tasks.append({
            'text': new_task,
            'date': today,
            'done': False,
            'priority': priority
        })
        save_tasks(tasks)
    return redirect('/')
```

Что изменилось:

- Добавлена строка `priority = request.form.get('priority', 'средний')`
- В `tasks.append()` добавлено поле `'priority': priority`

3.3. Отображаем приоритет в списке задач

Файл `templates/index.html`. Найдите строку вывода задачи и добавьте отображение приоритета (иконкой или текстом):

```

<ul>
  {% for task in tasks %}
    <li>
      {% if task.done %}
        <s>{{ task.text }}</s>
      {% else %}
        {{ task.text }}
      {% endif %}

      <!-- Отображение приоритета иконкой -->
      {% if task.priority == 'высокий' %}
        
      {% elif task.priority == 'средний' %}
        
      {% else %}
        
      {% endif %}

      - <small>{{ task.date }}</small>
      <a href="/toggle/{{ loop.index0 }}"> Выполнено</a>
      <a href="/edit/{{ loop.index0 }}">Редактировать</a>
      <a href="/delete/{{ loop.index0 }}">Удалить</a>
    </li>
  {% endfor %}
</ul>

```

3.4. Добавляем сортировку по приоритету

В `app.py` добавьте новый маршрут:

```

@app.route('/by_priority')
def by_priority():
    # Задаём числовые веса для сортировки (высокий = 3, средний = 2, низкий
    priority_order = {'высокий': 3, 'средний': 2, 'низкий': 1}

    # Сортируем задачи от высокого к низкому
    sorted_tasks = sorted(
        tasks,
        key=lambda task: priority_order.get(task.get('priority', 'средний'))
        reverse=True
    )
    return render_template('index.html', tasks=sorted_tasks)

```

Как это работает:

- Задаётся словарь `priority_order`, который переводит приоритет в число
- Функция `sorted()` сортирует задачи: чем больше число, тем выше приоритет
- `reverse=True` — сортируем от большего к меньшему (сначала высокие)

3.5. Добавляем ссылку на сортировку на главную страницу

Файл `templates/index.html`. Добавьте ссылку в блок фильтров:

```
<p>
  <a href="/"> Все задачи</a> |
  <a href="/active">✅ Активные</a> |
  <a href="/by_priority">🚦 По приоритету</a> |
  <a href="/done_all"> Выполнить все</a>
</p>
```

3.6. Добавляем возможность редактировать приоритет

Шаг 1. Изменяем шаблон `edit.html`. Добавьте выпадающий список:

```
<!DOCTYPE html>
<html>
<head>
  <title>Редактировать задачу</title>
  <style>
    body { font-family: Arial; margin: 40px; }
    input, select, button { padding: 8px; margin: 5px; }
  </style>
</head>
<body>
  <h1>Редактирование задачи</h1>

  {% if message %}
    <p style="color: red;">{{ message }}</p>
  {% endif %}

  <form method="POST">
    <input type="text" name="task" value="{{ task.text }}" required>

    <select name="priority">
      <option value="высокий"
        {% if task.priority == 'высокий' %}selected{% endif %}>
        🚦 Высокий</option>
      <option value="средний">
```

```

        {% if task.priority == 'средний' %}selected{% endif %}>
         Средний</option>
    <option value="низкий"
        {% if task.priority == 'низкий' %}selected{% endif %}>
         Низкий</option>
</select>

<button type="submit">Сохранить</button>
<a href="/">Отмена</a>
</form>
</body>
</html>

```

Шаг 2. Изменяем маршрут /edit в app.py:

```

@app.route('/edit/<int:task_id>', methods=['GET', 'POST'])
def edit_task(task_id):
    if task_id < 0 or task_id >= len(tasks):
        return "Задача не найдена", 404

    task = tasks[task_id]

    if request.method == 'POST':
        new_text = request.form.get('task', '').strip()
        new_priority = request.form.get('priority', 'средний')
        old_text = task['text']
        old_priority = task.get('priority', 'средний')

        if new_text == '':
            return render_template('edit.html', task=task, message="Текст н

        if new_text == old_text and new_priority == old_priority:
            return render_template('edit.html', task=task, message="Ничего

        task['text'] = new_text
        task['priority'] = new_priority
        save_tasks(tasks)
        return redirect('/')
    else:
        return render_template('edit.html', task=task)

```

4. Проверка работы

1. Запустите приложение: `python app.py`
2. Добавьте несколько задач с разными приоритетами
3. Проверьте, что иконки приоритета отображаются (🔴, 🟡, 🟢)
4. Нажмите «По приоритету» — задачи должны отсортироваться: сначала высокие, потом средние, потом низкие
5. Отредактируйте задачу — измените текст и/или приоритет, сохраните
6. Перезапустите сервер — все данные должны сохраниться

5. Задание для самостоятельной работы

1. Добавьте на страницу индикацию: сколько задач каждого приоритета (например, «Высоких: 3, Средних: 5, Низких: 2»)

Подсказка по реализации (в шаблоне `index.html`):

Воспользуйтесь фильтрами Jinja2 для подсчёта:

<p>

- 🔴 Высоких: `{{ tasks|selectattr('priority', 'equalto', 'высокий')|list|length }}`
- 🟡 Средних: `{{ ..... }}` |
- 🟢 Низких: `{{ ..... }}` </p>

`{{ tasks|selectattr('priority', 'equalto', 'высокий')|list|length }}` как это работает

`tasks` (несколько элементов)

↓

`selectattr('priority', 'equalto', 'высокий')` → оставляет только задачи с `priority = 'высокий'`

↓

`list` → превращает в обычный список

↓

`length` → считает количество

↓

вывод на экран это количество

Примечание. `'equalto'` Оператор сравнения (равно)

2. Добавьте маршрут `/by_priority_active`, который показывает только активные (невыполненные) задачи, отсортированные по приоритету

3. Оформите приоритеты цветом (например, красный для высокого, жёлтый для среднего, зелёный для низкого) через CSS

Подсказка. в index.html добавьте условие для применения класса

```
<li class="
  {% if task.priority == 'высокий' %}high
  {% elif task.priority == 'средний' %}medium
  {% elif task.priority == 'низкий' %}low
  {% endif %}
">
```

CSS:

```
.high {color: red; }
.....
или
.high {background-color: #d1df0cda;}
```